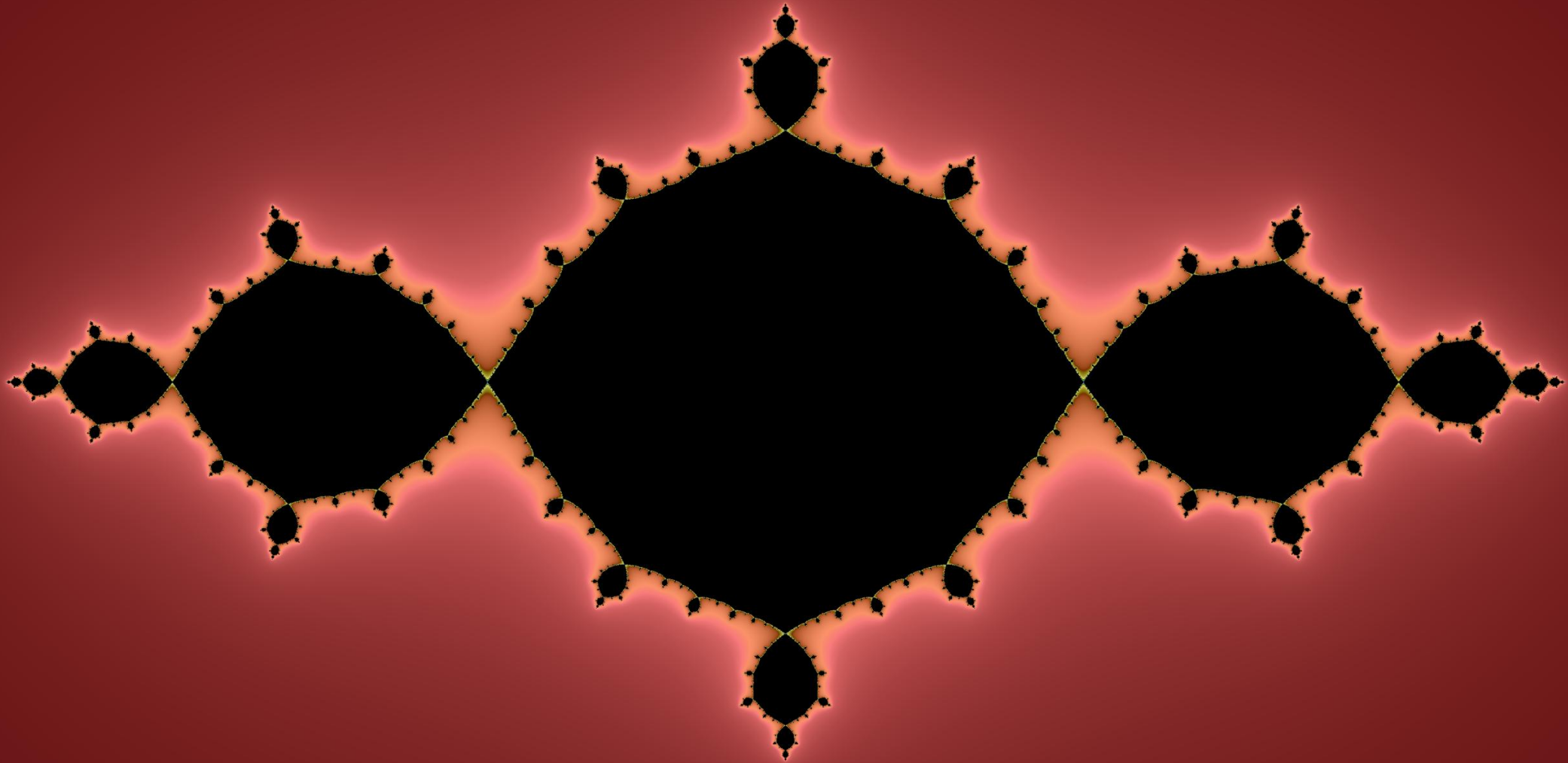


Fractal Software Design – Software Portability and Self Similarity (HAUD)

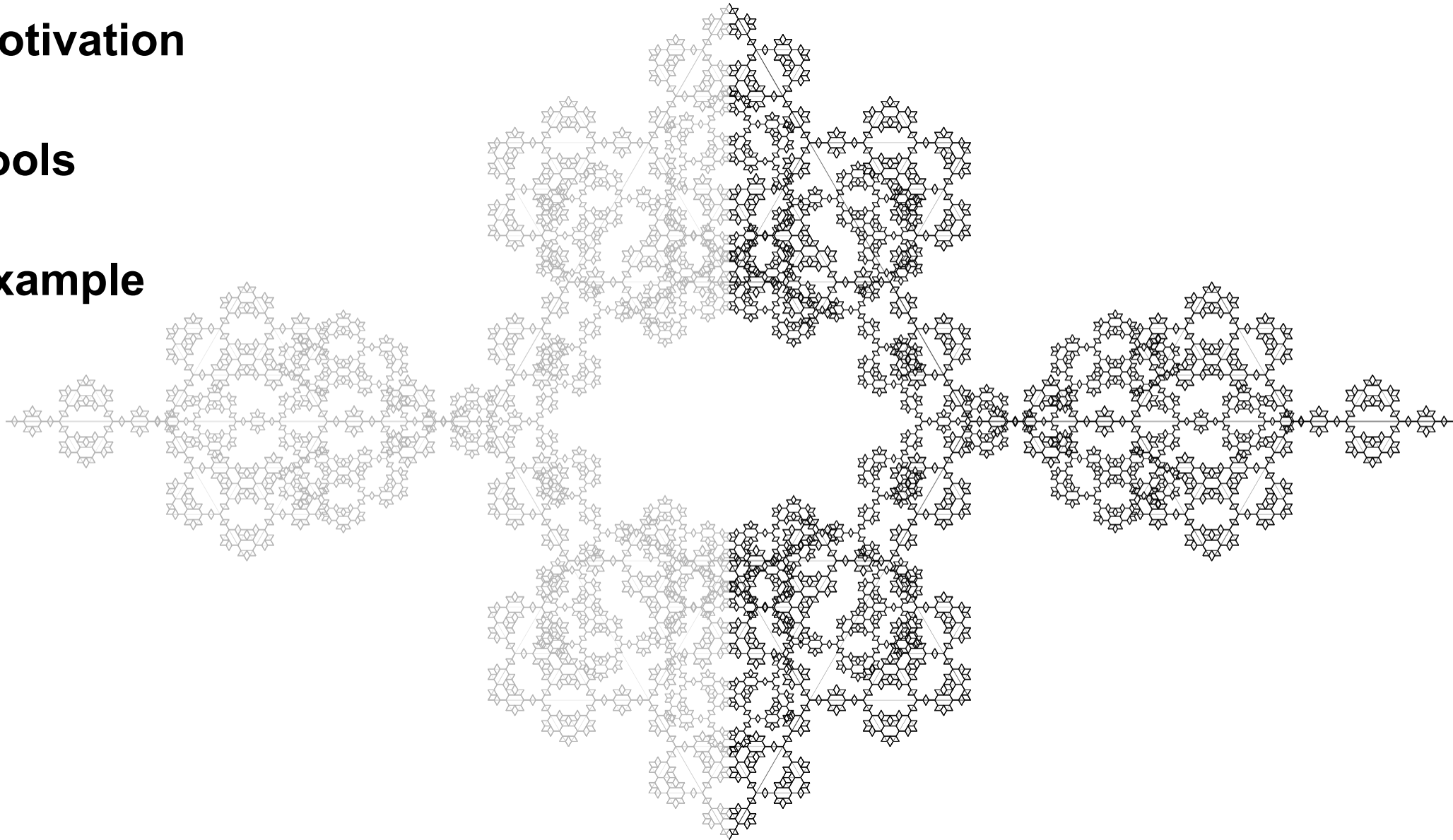
BECKHOFF



1. Motivation

2. Tools

3. Example



1. Motivation

- **Software portability**
 - Top level business logic shall not suffer from changes in lower abstraction levels.
- **Rigid hardware abstraction**
 - Unified software mechanics to shield business logic from hardware variance.
- **No God Objects**
 - Edge cases shall not disrupt abstraction.
- **Same cycle responsiveness**
 - Eliminate traditional, monolithic state machine design flaws.

1. Motivation

- **Software portability**
 - **Separation of business logic and motion logic enables portability.**
- **Changes in motion layer:**
 - Adding new features does not break the code.
 - Swapping a hardware controller does not break your business logic.
 - Introducing a new motion kernel does not break the code.

1. Motivation

- **Hardware Abstraction Layer**
 - **Strict interfaces enforce behavioural contract.**

Rigid = Enforced Contract = Safety

- **Rigid Interfaces ensure:**
 - Business logic never couples to hardware.
 - Hardware changes do not affect business logic.
 - Different hardware can be swapped transparently.

1. Motivation

- **NO God Objects**
 - **Work local, ship global.**
 - **Composition over inheritance.**
- **Composition ensures:**
 - Specialized logic can be reused.
 - Scales easily with future development.
- Know Thyself:
 - Know your job and do it well

1. Motivation

- **Same Cycle Responsiveness and Cycle Optimal Programming**
 - **Break the cycle.**
 - **Find a reusable structure to break safely.**
 - **Logic progression:**
 - Do not waste time to switch decision trees.
 - Ensure observability.
 - Keep decisions simple

1. Motivation

– **WHY?**

- **Moving at 10 m/s requires cycle optimal programming.**
- **Synchronizing place and time with software that is fast and safe.**
- **The machine must observe itself and tell the user about it.**
- **Communication agnosticism is a requirement for distributed motion applications.**
- **Top application layer must be oblivious to the hardware executing the commands.**

2. Tools

▪ SOLID DESIGN

- **S** — Single Responsibility Principle
 - a class should have one, and only one, reason to change.
- **O** — Open/Closed Principle
 - software entities should be open for extension but closed for modification.
- **L** — Liskov Substitution Principle
 - Objects of a superclass shall be replaceable with objects of its subclasses without breaking the application.
- **I** — Interface Segregation Principle
 - users should not be forced to depend upon interfaces that they do not use.
 - a software entity only sees the methods it actually needs to the work.
- **D** — Dependency Inversion Principle
 - Depend upon abstractions, not concretions.

2. Tools

▪ ACID DESIGN

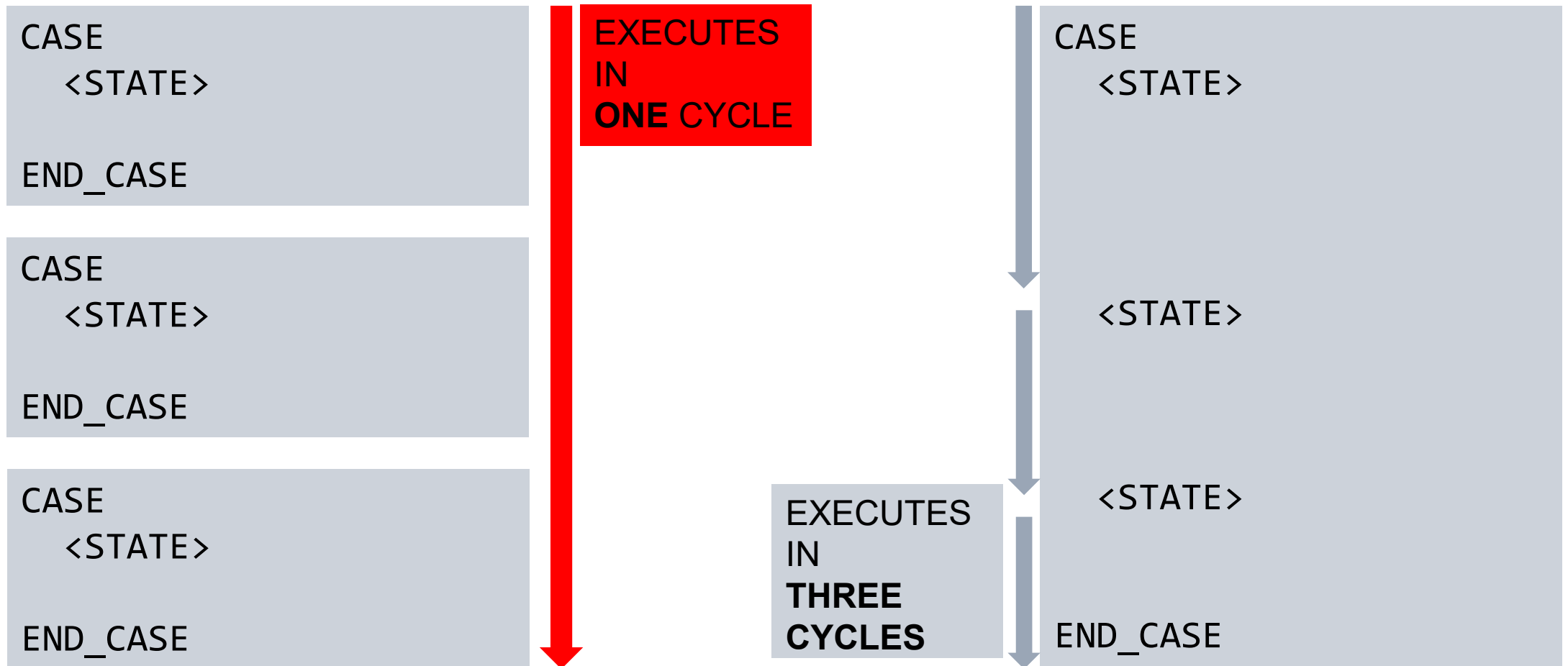
- **A** — Atomicity
 - A transaction is an "all-or-nothing" unit of work.
- **C** — Consistency
 - A transaction must bring the system from one valid state to another valid state.
- **I** — Isolation
 - Concurrent transactions should not interfere with each other.
- **D** — Durability
 - Once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

3. Examples

- **Break the cycle between states with E_PROGRESS.**
 - Core feature of the required Same Cycle Responsiveness.
 - Ensure atomicity in the context of the trusted cyclic RTS (TwinCAT).
 - Ensure machine readability without having to construct machine semantics on top.
 - Ensure human readability without translation of numbers into text.
 - Time Optimal Programming while keeping source code readable.

3. Examples

- Break the cycle between states with E_PROGRESS.



3. Examples

- Break the cycle between states with E_PROGRESS.

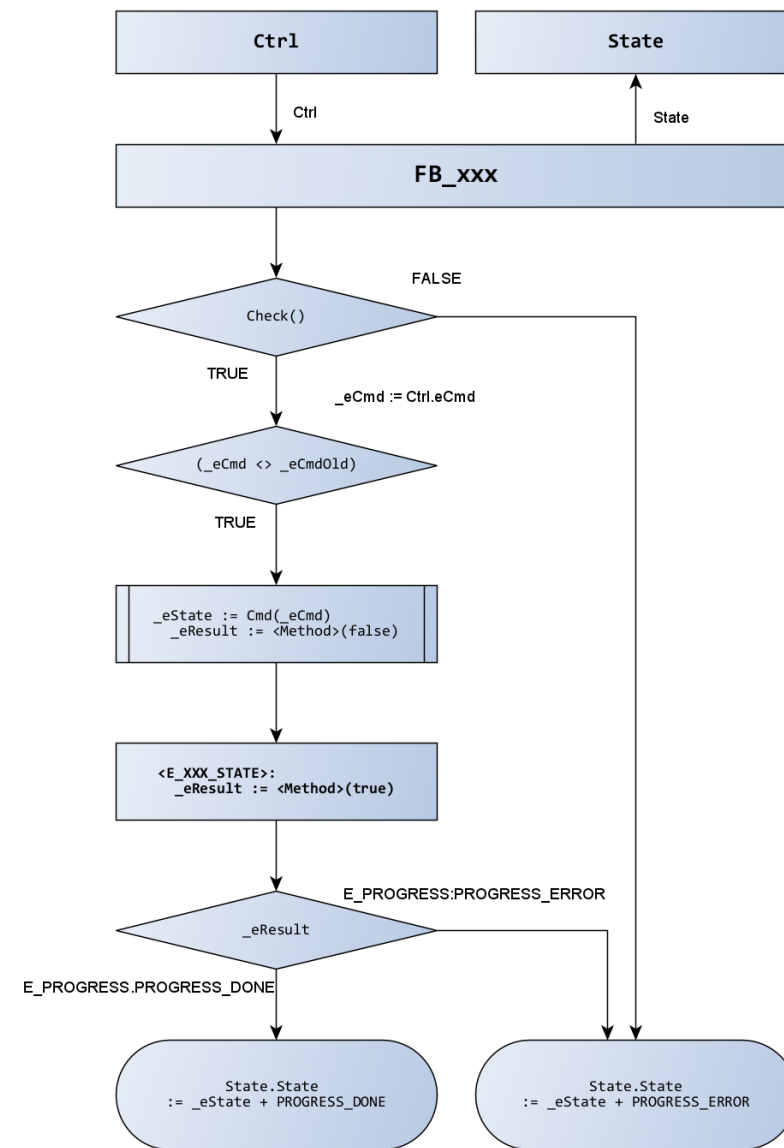
```
{attribute 'to_string'}  
TYPE E_PROGRESS :  
(  
    // progress has 2 use cases  
    // 1. as offset to cyclic interface's state  
    //    for the requested command/function  
    //    e.g. State := <Enum equivalent to eCmd> + E_PROGRESS  
    //  
    // 2. as state feedback (result) from (any) method  
    PROGRESS_INVALID,  
    PROGRESS_NOT_EXIST      := 100,  
    PROGRESS_INIT           := 1000,  
    PROGRESS_BUSY           := 2000,  
    PROGRESS_PREPARE        := 3000,  
    PROGRESS_STARTUP        := 4000,  
    PROGRESS_CHECK          := 5000,  
    PROGRESS_OCCUPIED       := 6000,  
    PROGRESS_WORKING        := 7000,  
    PROGRESS_STILL_WORKING  := 8000,  
    PROGRESS_ERROR          := 9000,  
    PROGRESS_DONE           := 10000  
)UINT;  
END_TYPE
```



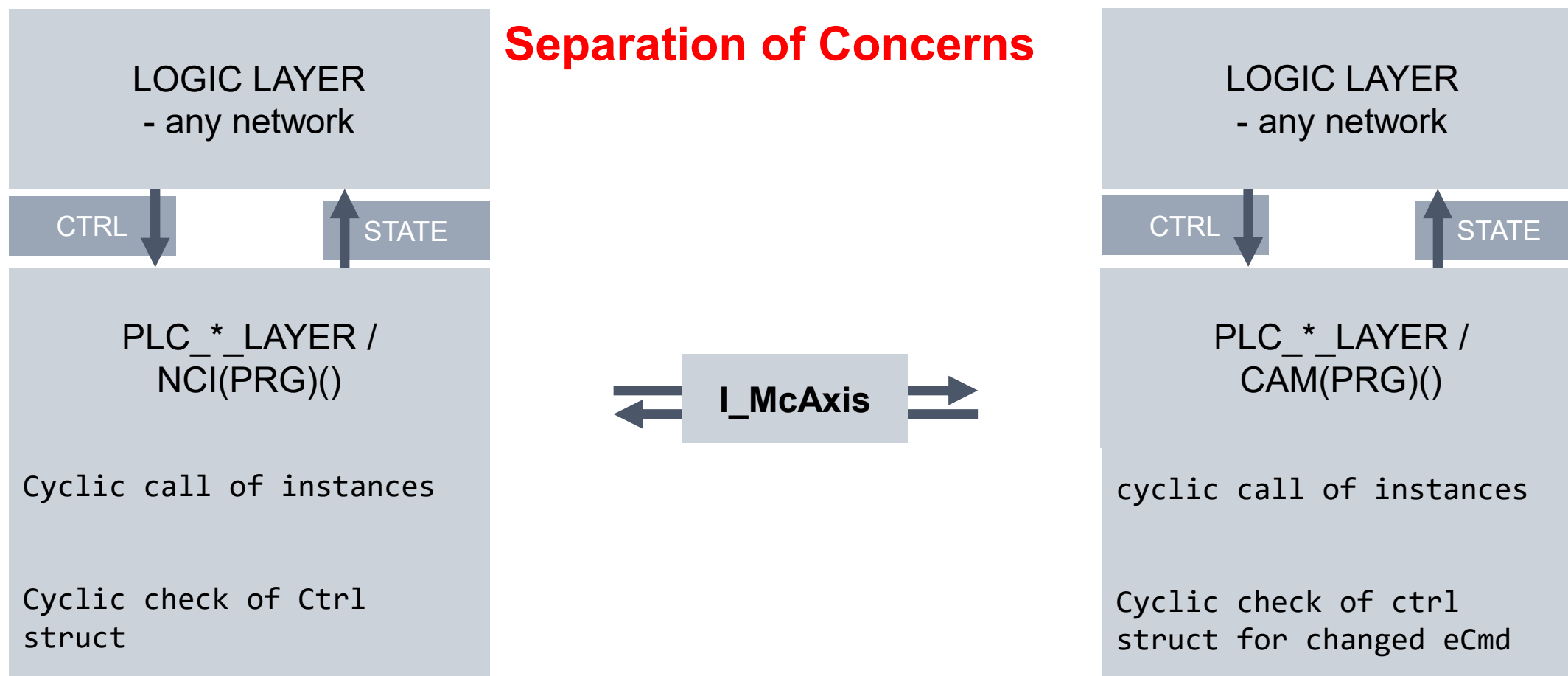
Shows
progress

3. Examples

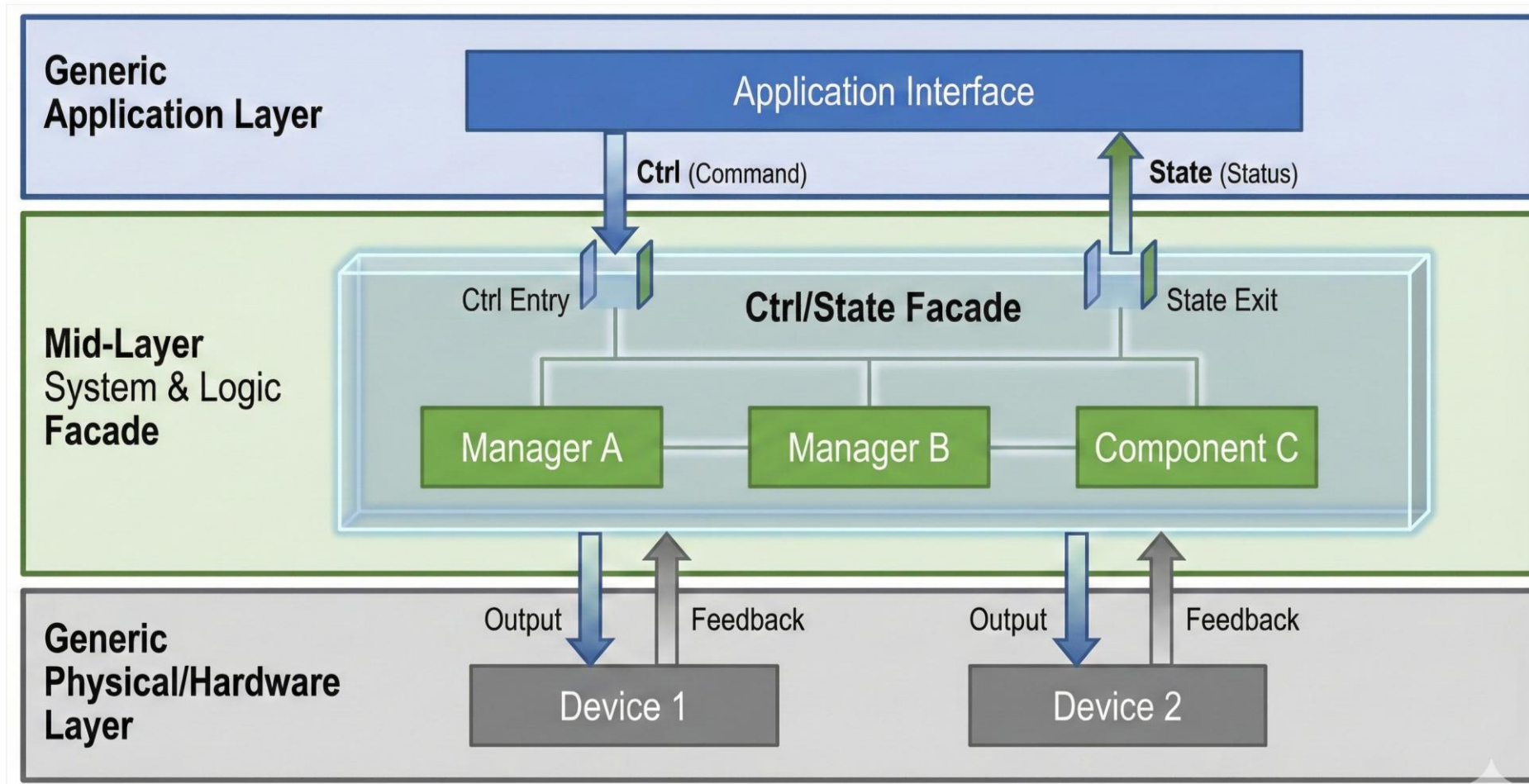
- **The Facade:**
 - State / Ctrl structures
 - Establishes unified access
 - Commands can simply be 'dropped' into Ctrl datafield
 - Enables asynchronous communication
 - Enables cyclic communication mapped onto any cyclic fieldbus
 - State is updated by own cyclic call
- State feedback for cyclic class wrappers
 - **Always** combined with E_PROGRESS
 - You can filter your response by a simple modulo division



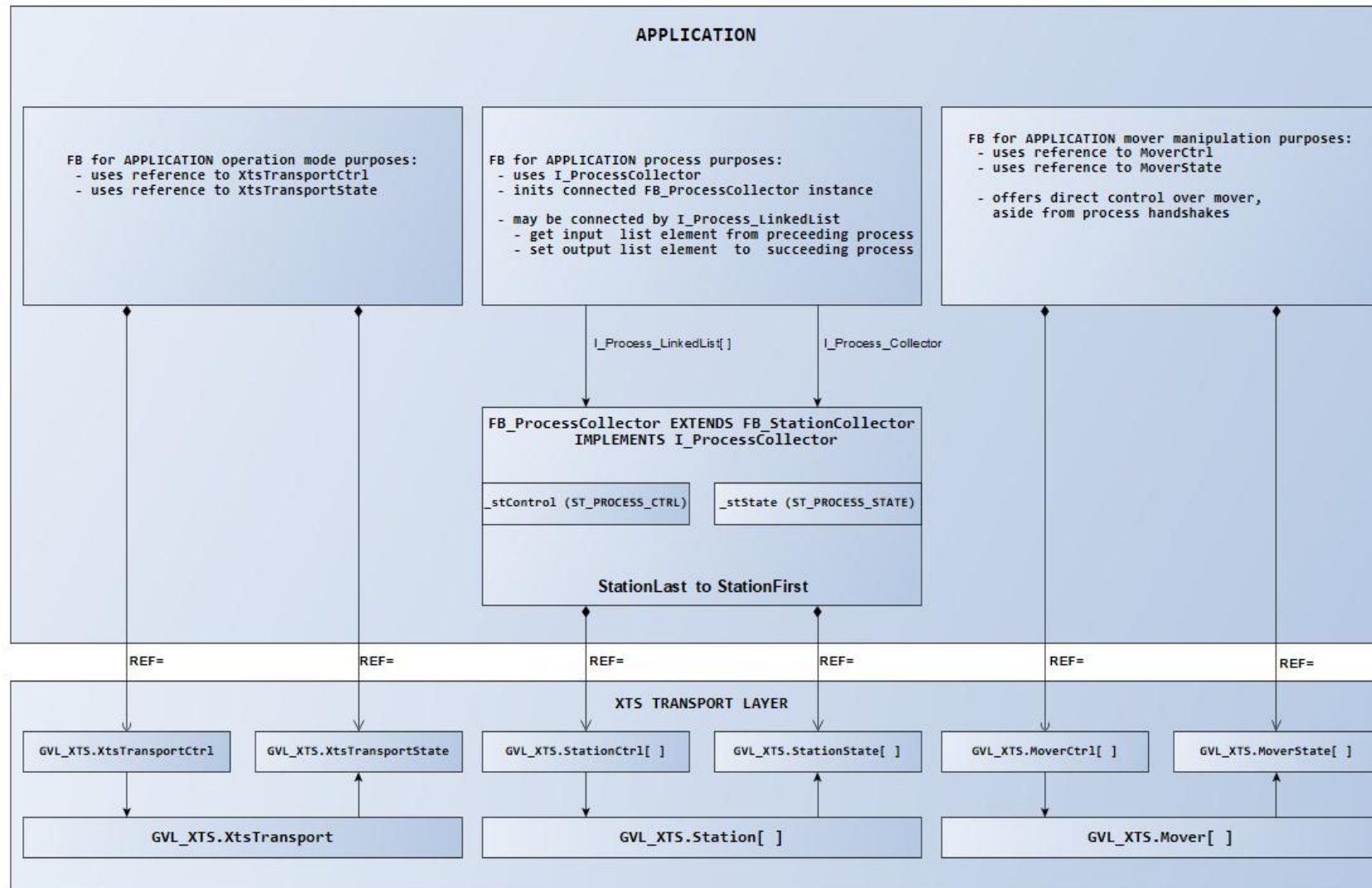
Software Design Example:



▪ Software Design Example: **The Facades**



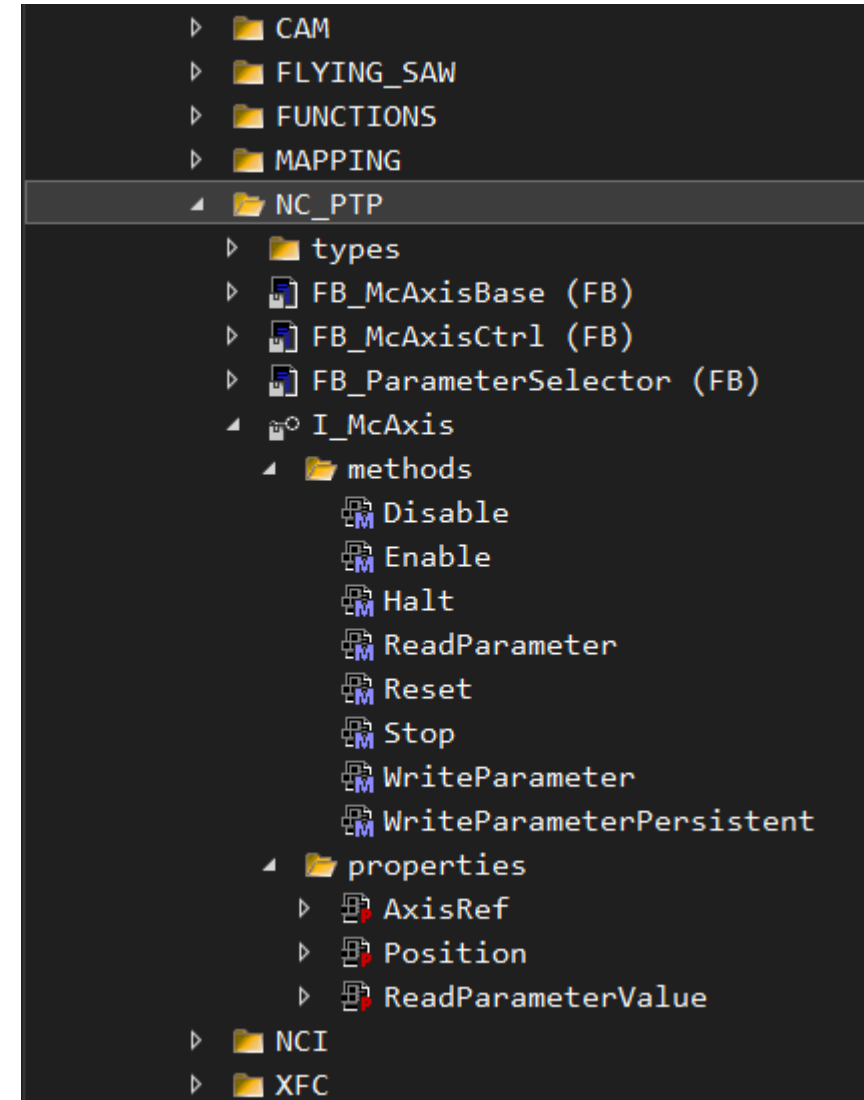
■ Software Design Example: **The Facades**



■ Software Design Example:

- Distributes only required method and properties.
- Enables strict separation of concerns and connects classes horizontally
- I_McAxis is the base interface and is not optional.

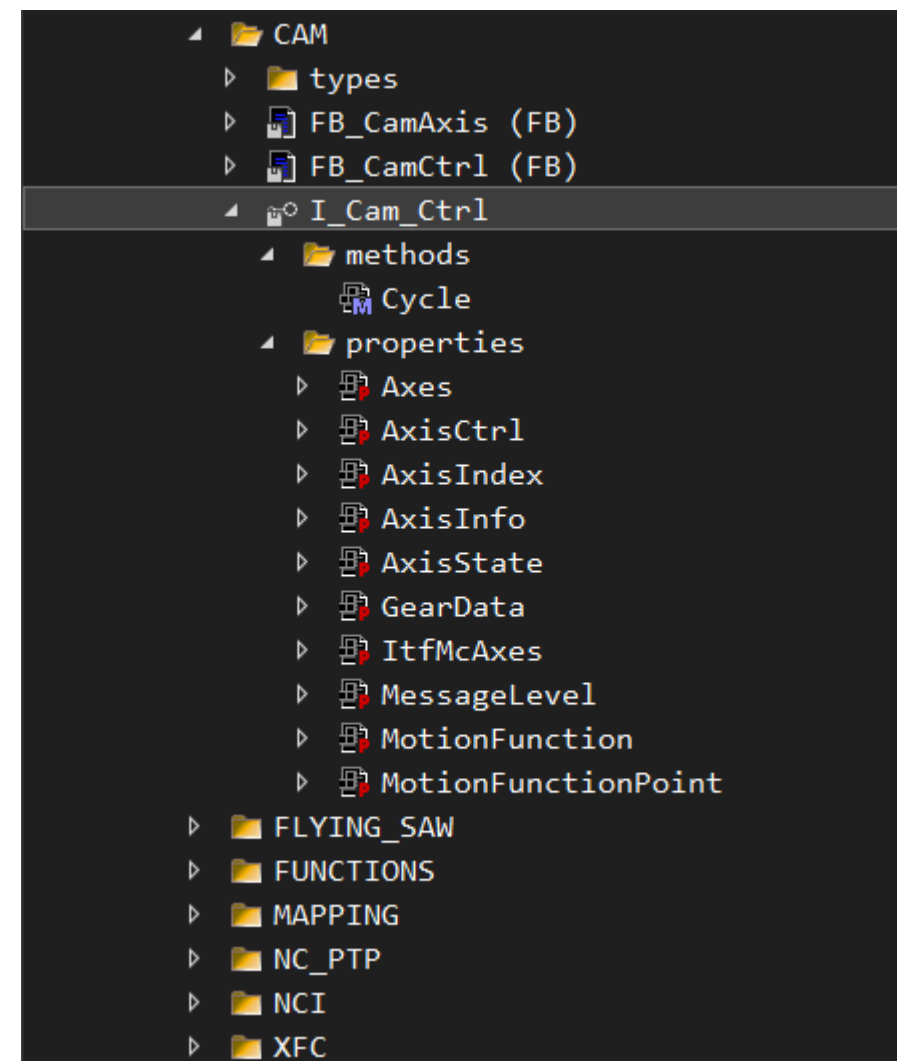
The Interface



■ Software Design Example:

The Interface

- Distributes only cycle method. The commands are written onto the façade structures.
- Enables use of empty interfaces for pre-compile defines
- Shall be void of any library datatypes from functions that are optional.



▪ Software Design Example:

The Interface

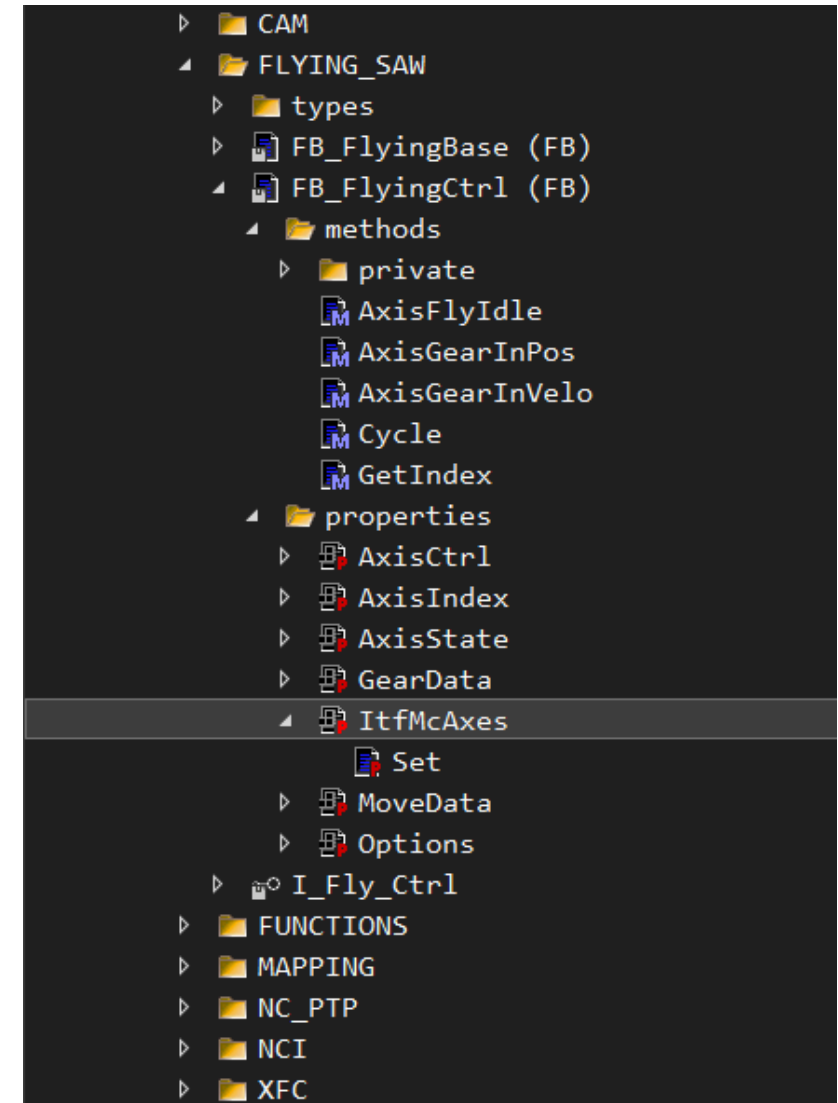
- Distributes only required method and properties.
- Enables use of empty interfaces for pre-compile defines
- Shall be void of any library datatypes from functions that are optional.

```
//-----  
// cyclic interface function block: FB_CamCtrl  
// - MC_Camming functions are here  
//-----  
{IF defined (CAM)}  
  {info 'PLC_MOTION_LAYER: CAM'}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF FB_CamCtrl;  
{ELSE}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF I_Cam_Ctrl;  
{END_IF}  
END_VAR
```

■ Software Design Example:

- Injected in class via property
- Class is oblivious to implementation details
- This is the contract on offer

The Interface



- **Public on GitHub:**

- [https://github.com/haud-ba/PLC MOTION LAYER](https://github.com/haud-ba/PLC_MOTION_LAYER)
- [https://github.com/haud-ba/XTS TRANSPORT LAYER](https://github.com/haud-ba/XTS_TRANSPORT_LAYER)
- [https://github.com/HAUDX/XTS TRANSPORT EXAMPLES](https://github.com/HAUDX/XTS_TRANSPORT_EXAMPLES)